CS3081 Cloud Computing: Mini Project Report

Author: Akanksha Gupta, 230101072

I trained a small language model to parse commands, then deployed it as a service on a local Kubernetes cluster. This report covers the seven project tasks, with measurements from my runs and supporting files in `results/`.

Task 1: Object Storage and Parallel Teacher Labelling

The teacher model (Phi-3.5-mini-instruct, served with Ollama at temperature 0, seed 42) was run over all 3,000 commands in `commands_pool.jsonl` to generate the four-key JSON labels. The frozen prompt is in `prompts/teacher_prompt.txt`. `worker/clean.py` then filtered the raw shard outputs into the training set `datasets/v1/train.jsonl` (SHA-256 `5b4a171a9c981bf8651a5077eacc6956f0e18a2ed34423011c8f1a9ba04a4f6d`).

Cleaning counts

The cleaning results account for all 3,000 input records:

Bucket	Count
Input records	3000
(a) Invalid JSON	0
(b) Wrong keys (≠ intent/date/time/place)	0
(c) Intent not one of the eight	12
(d) Slot value not `null` and not in the command	40
Duplicate commands (smallest id kept)	50
Kept	**2898**

The counts add up to the input total: `0 + 0 + 12 + 40 + 50 + 2898 = 3000`. All 3,000 teacher outputs were valid JSON.

Sharding

`commands_pool.jsonl` holds 3,000 commands, one per line, implicitly indexed by line number starting at 0. Labelling was parallelised across two workers, run as Kubernetes Jobs in the `pipeline` namespace, each reading two environment variables: `SHARD_COUNT=2` and a distinct `SHARD_INDEX` (0 and 1).

A worker processes exactly the lines whose index `n` satisfies `n % SHARD_COUNT == SHARD_INDEX`. Worker 0 therefore takes the even-numbered lines and worker 1 the odd-numbered lines. This assigns each command to one worker, with no gaps or overlap. To add workers, increase `SHARD_COUNT` and give each worker a different `SHARD_INDEX` from 0 to `SHARD_COUNT - 1`.

Each worker writes its own object, `datasets/teacher/shard-0.jsonl` and `datasets/teacher/shard-1.jsonl`, one line per command as `{"id", "text", "teacher_raw"}` with the teacher's reply stored exactly as generated. Writing to separate objects avoids write contention between workers.

On restart, a worker reads back its own shard object, skips the ids it has already labelled, and checkpoints to object storage every 50 commands. With the Jobs' `restartPolicy: OnFailure` and a `backoffLimit`, an interrupted worker can continue from its saved progress. This saved time because CPU labelling took several hours per shard.

After labelling, `clean.py` read all shards and accounted for 3,000 input records, matching the size of the command pool.

Task 2: Training the Student (Sequence-Level Distillation)

Setup

I fine-tuned `EleutherAI/pythia-70m` with PyTorch and Hugging Face Transformers on CPU, using the first 1,500 records of the cleaned `train.jsonl`. Each training example is built as:

```
Command: <the command>
JSON: <the teacher's four-key JSON reply>
```

The loss is computed only on the JSON completion. The prompt tokens (`Command: … JSON:`) are masked with label `-100`, so training focuses on generating the JSON response. This is sequence-level knowledge distillation: the teacher's generated text provides the training target.

All settings that affect the result are recorded in `training/config.yaml`:

Setting	Value
seed	42
learning rate	5e-4
batch size	8
epochs	v1 = 3, v2 = 5
max sequence length	128
libraries	torch 2.5.1, transformers 4.46.3, accelerate 1.15.0, tokenizers 0.20.3

The training commands are `make train` for v1 and `make train VERSION=v2` for v2. For a quick check, `make train SUBSET=50` runs the process on the first 50 records. The per-epoch training loss is written to `training/loss.csv` (and `training/loss_v2.csv`).

Training results

Both runs used a fixed seed and gradient clipping. The recorded loss decreased with each epoch:

Epoch	v1 loss	v2 loss
1	1.499	1.496
2	0.325	0.407
3	0.142	0.228
4	N/A	0.132
5	N/A	0.062

On CPU this took ≈ 9 min for v1 (3 epochs) and ≈ 15.5 min for v2 (5 epochs) at about 8.4 examples/s.

Evaluation

I scored each model with `tools/score.py` on `validation.jsonl`, which contains 100 held-out commands that were not used for training. Raw predictions and score reports are in `results/`.

Metric (%)	Teacher (Phi-3.5)	Base (untrained Pythia-70m)	Student v1 (3 ep)	Student v2 (5 ep)
json_validity	100	0	**100**	100
intent_accuracy	74	0	**77**	71
date_accuracy	85	0	75	80
time_accuracy	87	0	84	88
place_accuracy	92	0	85	83
full_frame	52	0	**41**	41

The untrained base model produced no valid JSON and scored zero on every metric in this evaluation. After training, v1 reached 100% JSON validity and 41% full-frame accuracy. Its intent accuracy was 77%, compared with 74% for the teacher, which is about 54 times larger. However, v1 was less accurate on date and place extraction, at 75% and 85% compared with the teacher's 85% and 92%. These slot errors contributed to the lower full-frame score of 41%, compared with 52% for the teacher. On this validation set, the student reached about 79% of the teacher's full-frame accuracy.

Training v2 for two extra epochs reduced the final training loss from 0.142 to 0.062, but full-frame accuracy stayed at 41%. Individual slot scores changed, and intent accuracy fell from 77% to 71%. The lower training loss without a full-frame improvement suggests overfitting. I chose v1 for the main deployment because it had the same full-frame score, higher intent accuracy, and a shorter training time. I kept v2 for the rolling update test in Task 4.

Both students meet the required validation thresholds: JSON validity 100% (≥ 95% required) and full-frame 41% (≥ half the teacher's 52%, i.e. ≥ 26%).

Model artefacts

Both versions' model and tokenizer files are stored in object storage under the `models` bucket (`models/v1/`, `models/v2/`). The inference service downloads them at start-up in Tasks 3 and 4. A SHA-256 checksum of each release archive is recorded in `models/v1/model.sha256` and `models/v2/model.sha256`:

- v1: `b3c7fa655d84442755426e9facb5f8f82768bf784ff40cf9c67295954b8b2681`
- v2: `ea9a1e85006537e9d010a67f1f93709f32ec69001faec8886c7975a905c31e16`

v1 is also published as a GitHub Release asset (`pythia70m-v1.tar.gz`), whose checksum matches `models/v1/model.sha256`. Model weights are never committed to Git or included in the container image. Only the small checksum files are tracked.

Sequence-level vs logit-based distillation

Sequence-level distillation uses the teacher's generated JSON as the training target, with next-token cross-entropy as the loss. It only requires the teacher's final text output.

Logit-based distillation trains the student to match the teacher's probability distribution over tokens at each position. It commonly uses KL divergence between their softmax outputs, with a temperature setting. This requires the teacher's per-token logits as well as compatible token distributions.

I could not use logit-based distillation in this setup for two reasons:

1. The teacher (Phi-3.5) is served through Ollama's generation API, which returns the generated text, without the per-token logits needed for this loss.
2. The teacher and student use different tokenizers and vocabularies. Their probability vectors refer to different token sets and would need explicit vocabulary alignment before comparison.

A limitation of sequence-level distillation is that each command has just one teacher answer as its target. The student receives no information about the teacher's uncertainty or alternative answers, and incorrect labels can become training targets. With only 1,500 training examples, label quality and coverage are concerns. The validation results show lower date and place accuracy than the teacher, although these runs do not isolate the cause of those errors.

Task 3: The Inference Service in a Container

The service (`service/app.py`) is a FastAPI app on port 8000 with three endpoints: `POST /parse` (returns the four-key JSON and nothing else, plus the headers `X-Model-Version` and `X-Served-By`), `GET /healthz` (200 while the process is alive), and `GET /readyz` (503 until the model has loaded, 200 after). All configuration uses `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, and `MAX_NEW_TOKENS`, with credentials supplied through a Kubernetes Secret. At start-up, the service downloads the model from object storage. It validates each model output against `schema.json` and returns HTTP 500 with `{"error":"invalid_model_output"}` if validation fails. Setting `FAULT_INJECT=1` replaces the output with `not json` to test this error handling.

The image uses pinned dependencies, contains no GPU libraries or model weights, and is 1.6 GB, below the 2 GB limit. I ran it with a 1 CPU / 1 GiB limit and sent 50 requests in sequence. All 50 succeeded, peak memory was 424 MiB, and there was no out-of-memory kill. With `FAULT_INJECT=1`, `/parse` returned HTTP 500 with the expected error body. The results are in `results/task3_container_test.md`.

Task 4: Kubernetes Deployment, Scaling, and Reliability

The service is deployed in namespace `app` as a Deployment (2 replicas), a Service, and a Traefik Ingress that routes `/parse`; an HPA scales 1-2 replicas on 70% CPU. The container has CPU/memory requests and limits (`requests cpu 100m / mem 512Mi`, `limits cpu 1 / mem 1Gi`), a liveness probe on `/healthz`, and a readiness probe on `/readyz`. A Pod does not become a Service endpoint until its model has loaded. The Ingress exposes `/parse` at `http://localhost:8080`.

The deployment tests gave the following results (details in `results/task4_evidence.md`):

- **Autoscaling:** at rest 1 replica; under the standard load CPU hit 999% of the 100m request and the HPA scaled **1** → **2**, then back to **1** when the load stopped.
- **Load balancing:** with two replicas the two Pods served 52% / 48% of the successful requests (both ≥ 30%).
- **Rolling update:** changing `MODEL_VERSION`/`MODEL_URI` from v1 to v2 under load shifted traffic v1 → v2 with **0% failed requests**; `kubectl rollout undo` returned `X-Model-Version` to v1. The rollout used a `preStop` drain and `maxUnavailable: 0` to allow requests to finish during the update.
- **Recovery:** deleting a Pod under load, Kubernetes restored two Ready replicas in **~21 s** with **0% failed requests**.
- **Cold start:** deleting all Pods three times, the median time from Pod creation to Ready was **7 s**, and the Pod logs show each new Pod downloading the model from object storage.

The cluster runs as a single k3d node on one laptop, with scaling configured for 1-2 replicas. Every replica shares the same physical host, so these tests do not demonstrate recovery from a machine failure.

Task 5: Subnetting and Network Access Control

I divided `10.20.0.0/16` into four contiguous, aligned subnets, starting with the largest host requirement. The calculations and host ranges are in `network/subnets.md`.

Subnet	Hosts	Network	Prefix	Broadcast	Usable
app	1,000	10.20.0.0	/22	10.20.3.255	1022
public	250	10.20.4.0	/24	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.159	30

The NACL for the public subnet (`network/nacl.json`) denies the abusive range `198.51.100.0/24` first, using a lower rule number than the web-port allow rules. It allows TCP 80/443 from anywhere, allows SSH only from the management subnet, allows the app subnet's replies inbound, and outbound allows TCP 8000 to the app subnet plus client replies on ephemeral ports. All six sample packets passed the checker (`python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check`).

A stateless NACL checks each packet independently, so reply traffic needs explicit rules too. Without them, responses would be dropped. The first matching rule wins: the `198.51.100.0/24` deny must have a lower number than the 80/443 allow, or a packet from that range to port 443 would be allowed before the deny is ever reached.

Task 6: Reproducibility, Testing, and CI/CD

`make up` builds and deploys the system from a fresh clone. It runs `scripts/deploy.py` to create the k3d cluster and Secrets, deploy SeaweedFS, download the model from the GitHub Release into object storage, build and import the image, and deploy the app. `make down` deletes the cluster.

The GitHub Actions workflow (`.github/workflows/ci.yml`) runs ruff linting, then 12 pytest tests covering output validation, cleaning rules, and subnet calculations. It downloads the model from the GitHub Release, evaluates it on `validation.jsonl`, and applies a quality gate with `score.py --min-validity 95`. A `workflow_dispatch` input selects the untrained Pythia-70M to test whether the gate rejects it. The gitleaks scan reported no secrets. `AI_USE.md` records the AI tools I used and how I checked their output.

Quality-gate failure run using the untrained model.
https://github.com/AkankshaGupta259/Command-Parsing-Language-Model/actions/runs/42506046564/job/128808877262

Task 7: Performance, Cost, and Review

I ran the standard load test (`--users 4 --spawn-rate 4 --run-time 2m`) with two Ready replicas. The Locust output is in `results/`, with details in `results/task7_performance.md`.

Metric	Value
Latency p50 / p95	2.4 s / 5.2 s (p90 4.7 s)
Throughput	1.49 req/s
Error rate	1.14% (service correctly returning 500 for invalid model output)
CPU / memory per Pod	~0.9 core, ~650 MiB (limits 1 core / 1 GiB)
Cold start	7 s median

The measured p95 was 5.2 s, slightly above the 5 s target. Repeated runs with the same settings gave p95 values of 5.1-12 s, with occasional requests taking about 12 s. CPU throttling on the laptop appeared to contribute to this variation. Testing on hardware with more available CPU would help establish how much of the delay came from resource contention.

Cost (`rate_card.json`, INR). Each replica is billed for its CPU and memory limits per hour: `1 vCPU × 3.50 + 1 GiB × 0.45 = 3.95` per replica-hour, so two replicas cost 7.90/hour. At 1.49 req/s that is 5,364 requests/hour, giving `cost per 1,000 = (7.90 / 5,364) × 1,000 =` **₹1.47**.

SLO. The target was for 99% of `/parse` requests to return HTTP 200. In the 176-request load-test run, two requests failed, giving a success rate of about 98.86%. The error budget was `1% × 176 = 1.76` requests, so the two failures used about 114% of the budget. Both were HTTP 500 responses for malformed model output. The service rejected those outputs as designed, but they still counted as failures against the SLO.

Review.

Component / concern	Evidence (measured)	Limitation I found	Improvement I propose
Data and storage	3,000 teacher labels → 2,898 kept; reproducible `train.jsonl` checksum; teacher validity 100%	Teacher labels noisy (52 bad intent/slots, 50 duplicates)	Larger, cleaner command pool and a tighter teacher prompt
Model quality	Student full-frame 41% vs teacher 52%, base 0%; validity 100%	Weak on slot extraction (date 75% vs 85%); v2 showed signs of overfitting	More training data; explore logit-based distillation with logits and vocabulary alignment
Latency and throughput	p50 2.4 s, p95 5.2 s, 1.49 req/s at 2 replicas	CPU-bound single core per Pod; heavy tail under contention	More CPU per Pod, batching, or a GPU; cap generation length
Resource utilisation	~0.9 core and ~650 MiB per Pod under load	Pods saturate their core at 4 users	Raise CPU limit, or the HPA ceiling on a bigger cluster
Reliability and recovery	Rolling update 0% fail; recovery ~21 s, 0% fail; cold start 7 s	Single node; no cross-machine fault tolerance	Multi-node cluster
Security and cost	gitleaks clean; creds only from a Secret; ₹1.47 per 1,000 requests	Replicas billed on limits even when idle	Scale to zero at idle, or right-size the limits